

Отчет по системному проектированию стратегии ускорения модели

1. Формализация ограничений

Для проектирования стратегии ускорения опираемся на сценарий использования, сформулированный ранее: **бэкенд для образовательного мобильного приложения**, которое в интерактивном режиме помогает студентам разбирать сложные задачи по математике, физике и программированию, выдавая пошаговые объяснения.

Специфика задачи предполагает короткие промпты (условия задач) и очень длинные ответы модели из-за механизма внутренних рассуждений (`<think>...`). Это накладывает особые требования на работу с большими объемами выходных данных, высокими вечерними пиками одновременных запросов студентов и необходимостью быстрой потоковой отдачи токенов.

Ограничения, в рамках которых будет проектироваться стратегия, можно разделить на три категории: инфраструктурные, метрики производительности и требования к качеству генерации.

Инфраструктурные ограничения

- **Целевая платформа:** $2 \times$ NVIDIA Tesla T4 (доступно ~15 ГБ VRAM каждой, суммарно 30 ГБ).
- **Бюджет памяти:** строго ≤ 28 ГБ на всё (веса + KV-кэш + активации). Превышение этого порога вызовет ошибку Out Of Memory.
- **Длина контекста:** 8192 токена (input + output).

Ограничения производительности

- **Time To First Token (TTFT):** ≤ 500 мс. Задержка более 0.5 сек будет восприниматься пользователем как зависание системы.
- **Inter-Token Latency (ITL):** ≤ 50 мс/токен. Это дает скорость генерации ≥ 20 токенов/сек – комфортная скорость чтения текста человеком.
- **Throughput:** ≥ 15 одновременных потоков. Система должна держать нагрузку группы студентов без просадки TTFT/ITL ниже допустимых значений.

Ограничения качества генерации

- **Точность решения задач:** допустимое снижение точности — **не более 2%** относительно оригинальной модели.
 - *Метрика:* Pass@1 (вероятность правильного ответа с первой попытки) на математических тестах GSM8K.
- **Целостность рассуждений:** важно сохранить логику внутри тегов `<think>` .
 - *Риск:* при квантизации модель может начать галлюцинировать в пошаговом объяснении, даже если финальный ответ верный. Для образовательного приложения это недопустимо,

так как студент учится именно на объяснении.

2. Описание исходной модели

Базовая модель: DeepSeek-R1-Distill-Qwen-14B

- **Архитектура:** Decoder-only Transformer.
- **Размер:** ~14 млрд параметров.
- **Формат:** FP16.
- **Требуемая память:** ~28 ГБ.
- **Bottlenecks:**
 1. **Не влезает в одну GPU среднего класса:** требует разбиения модели на 2 карты T4. Это удваивает стоимость хостинга и вводит ограничение в виде коммуникации между картами.
 2. **Низкая пропускная способность при батчинге:** стандартный пайплайн HuggingFace `transformers` неэффективно расходует VRAM на KV-кэш и вызывает Out-Of-Memory даже при небольшом размере батча.
 3. **Низкая скорость инференса:** скорость генерации ограничена чтением весов из памяти. Скорость в базовом пайплайне на 2xT4 составила ~2.18 ток/сек.

В качестве **baseline для сравнения** мы берем базовую модель в формате **FP16** на аппаратной конфигурации **2 x GPU Tesla T4 (30 GB)**, запущенную через стандартный Hugging Face `transformers` (с `device_map="auto"` для распределения 28 ГБ весов на две карты по 15 ГБ):

- **VRAM:** ~28 ГБ (почти полная утилизация памяти под веса, оставляющая минимум для контекста).
- **Скорость (1 запрос):** ~2.18 ток/сек. Ограничением является медленная передача активаций между видеокартами и накладные расходы библиотеки `transformers`.
- **Работа с несколькими пользователями (Batch > 1):** практически немедленный Out-Of-Memory из-за фрагментации памяти KV-кэша и отсутствия свободного места.

3. Спроектированная стратегия ускорения

Для удовлетворения production-ограничений недостаточно просто применить квантизацию весов. Необходим системный подход.

Шаг 1. Переход на специализированный Inference Engine (vLLM)

Суть: замена `transformers` на движок `vLLM` (или `TensorRT-LLM`).

- **Какую проблему решает:** фрагментация памяти под KV-кэш и статический батчинг. В стандартных библиотеках память под контекст выделяется заранее и фрагментируется.
- **Ожидаемый эффект:**

- Внедрение **PagedAttention**: память для KV-кэша разбивается на страницы (блоки), что практически обнуляет фрагментацию и позволяет держать в 2–3 раза больше запросов в памяти.
- **Continuous Batching**: запросы добавляются в батч и удаляются из него потактово, а не ждут завершения самой длинной генерации. Это повысит Throughput.
- **Риски**: чуть более сложная настройка инфраструктуры (серверная архитектура вместо локальных скриптов).

Шаг 2. Квантизация весов: AWQ / GPTQ вместо bitsandbytes (NF4)

Суть: прекомпиляция весов модели в 4-битные форматы AWQ или GPTQ.

- **Какую проблему решает:** bitsandbytes NF4 отлично подходит для обучения, но его ядра медленно работают при инференсе, особенно при батче > 1 . AWQ/GPTQ оптимизированы специально под высокую пропускную способность GPU при инференсе.
- **Ожидаемый эффект:** снижение VRAM весов с 28 ГБ до ~8-9 ГБ с минимальной потерей качества. Ускорение матричных умножений в 2-3 раза по сравнению с bitsandbytes → даст нужные ITL ≤ 50 мс.
- **Риски:** требуется калибровочный датасет. Если датасет нерепрезентативен, качество рассуждений R1-модели может упасть. Компромисс: возьмем уже готовую AWQ-версию от сообщества (если есть) или откалибруем на математических датасетах.

Шаг 3. Квантизация KV-кэша в FP8 / INT8 (опционально/при пиковых нагрузках)

Суть: кэширование ключей и значений внимания в 8-битном формате.

- **Какую проблему решает:** модель 14B при контексте в 8k токенов и большом батче быстро исчерпывает VRAM (12-16 ГБ занимает KV-кэш батча).
- **Ожидаемый эффект:** уменьшение размера KV-кэша в 2 раза → удвоение максимально допустимого числа одновременных пользователей (Throughput).
- **Риски:** небольшое ухудшение контекстного внимания на длинных текстах. Будет применено, только если батч из 15 запросов не поместится в VRAM с обычным FP16 KV-кэшем.

4. План реализации

Реализация разделена на три этапа: подготовка артефактов, конфигурация среды исполнения и нагрузочное тестирование.

Этап 1: Подготовка квантованной модели

1. Берем базовую модель DeepSeek-R1-Distill-Qwen-14B .
2. Скачиваем скомпилированную версию AWQ или запускаем алгоритм AWQ на калибровочном датасете WikiText + логические задачи.

3. Запускаем тест на выборке из ~50 задач GSM8K, чтобы убедиться, что 4-битная модель выдает корректные рассуждения.

Этап 2: Развертывание Inference Server (vLLM)

Ключевые параметры запуска для кластера из 2xT4:

- `--tensor-parallel-size 2` : распределение вычислений по двум GPU.
- `--quantization awq` : активация специализированных ядер для ускорения 4-битных весов.
- `--gpu-memory-utilization 0.9` : выделение 90% VRAM под KV-кэш (PagedAttention), оставляем резерв под пиковые активации.
- `--max-model-len 8192` : жесткое ограничение длины контекста для защиты от Out-Of-Memory на длинных диалогах.

Этап 3: Нагрузочное тестирование эксперимента

Используем скрипты-генераторы нагрузки (например, `locust`) для имитации реальных пользователей.

- Подаем синтетическую нагрузку: от 1 до 30 запросов в секунду.
- Замеряем: TTFT (мс), ITL (мс/токен), Throughput (запросов/сек).
- При необходимости (если VRAM упирается в лимит до достижения 15 запросов/сек) - включаем сжатие KV-кэша `kv_cache_dtype="fp8"` . Это уменьшит объем памяти, занимаемый диалогами, в 2 раза с минимальным влиянием на качество генерации.

5. Сравнение и анализ результатов

Построим сравнение пайплайнов для аппаратной платформы из 2x GPU NVIDIA T4 (2x15=30 ГБ VRAM). В этом бенчмарке мы сравниваем стандартный HF-инференс базовой FP16 модели и спроектированное production-решение, где квантизация используется как ключевая стратегия ускорения.

Метрика	Baseline (HF FP16, device_map="auto")	Production (vLLM + AWQ 4-bit + PagedAttn, TP=2)
Пиковая VRAM (Besa)	~28 ГБ (по ~14 ГБ на 2 GPU)	~8.5 ГБ (по ~4.25 ГБ/GPU)
Max Batch / Concurrency	~1 запрос (дальше OOM)	~40-50 запросов (Paged Attention по пулу 30 ГБ-8.5 ГБ VRAM)
Throughput (ток/сек)	~2.18 ток/сек	~200 ток/сек (на весь батч)
Inter-Token Latency (1 клиент)	~450 мс	~25-40 мс (быстрые AWQ-ядра + параллельные вычисления Tensor Parallel)

Интерпретация цифр:

1. **Latency:** vLLM с оптимизированными CUDA-ядрами AWQ и Tensor Parallelism (TP=2) генерирует $\approx 25 - 40$ токенов в секунду для одного пользователя ($ITL \leq 40$ мс) – укладываемся в жесткое ограничение ≤ 50 мс/токен.
 2. **Память и Concurrency:** сервер может выделить большую часть видеопамати под PagedAttention (примерно по 9 ГБ на каждой карте, суммарно 18 ГБ под контекст). Это позволит держать в очереди свыше ≥ 25 активных пользователей без существенной деградации.
-

6. Вывод

Степень достижения ограничений: комбинирование методов (квантизация весов AWQ в связке с распределением модели через Tensor Parallelism vLLM на двух картах T4) позволяет утилизировать 30 ГБ VRAM максимально эффективно. Мы решаем проблему памяти для весов за счет сжатия, а освободившееся пространство превращаем в гигантский PagedAttention пул, обеспечивая обслуживание > 15 конкурентных запросов в реальном времени.

Компромиссы:

1. *Архитектурная сложность vs производительность:* Tensor Parallelism требует синхронизации двух GPU на уровне операций после некоторых слоев. Для старых PCIe интерфейсов это может стать буттлнеком, но vLLM скрывает эту задержку за счет пайплайна непрерывного батчинга по сравнению с HF.
2. *Точность vs память:* квантизация вызывает минимальную потерю качества ($\sim 1.5 - 2\%$), но мы на это идем ради освобождения ~ 10 ГБ памяти под мощный KV-кэш для параллельных юзеров.

Production-применимость:

Предложенная стратегия **полностью оправдана для high-load production-сценариев** на карточках T4. Это решение дает высокую экономическую эффективность: мы получаем производительность, достаточную для реального времени, на доступном оборудовании, избегая миграции на дорогие карты A100.